

This document defines the agreed structure and shared API for each class so that everyone's code remains compatible. The required method names, parameters and return types should remain consistent. How each class is implemented internally is up to the person responsible for it. If you believe the shared structure should change, discuss the change with the team before modifying it.

CELL.JAVA – IMPLEMENTATION SPECIFICATION

Purpose

`Cell` represents one location/square in the forest grid.

A Cell is responsible for:

- Its position in the forest.
- Whether it contains a Tree.
- Which Tree it contains.
- How strongly wind affects that location.

Class

Class name: `Cell`

No inheritance or interface is required.

Fields

`int x`

Horizontal position in the forest grid.

`int y`

Vertical position in the forest grid.

`Tree tree`

The Tree occupying this Cell. A Cell may contain no Tree.

`double windExposure`

How strongly wind affects this Cell.

Suggested range:

0.0 = no wind influence
1.0 = maximum wind influence

Constructor

```
Cell(int x, int y)
```

Parameters:

x – horizontal position

y – vertical position

A new Cell should know its coordinates, initially contain no Tree, and have an appropriate default wind exposure.

Relationships

- Forest HAS many Cells.
- Cell HAS a Tree.
- Wind updates the Cell's wind exposure.

Required Methods

```
boolean hasTree()
```

Checks whether the Cell contains a Tree.

```
Tree getTree()
```

Returns the Tree stored in the Cell.

```
void plantTree(Tree tree)
```

Places the supplied Tree into the Cell.

```
int getX()
```

Returns the Cell's horizontal coordinate.

```
int getY()
```

Returns the Cell's vertical coordinate.

```
double getWindExposure()
```

Returns the current wind influence on the Cell.

```
void setWindExposure(double windExposure)
```

Updates the Cell's wind exposure.

Responsibility Boundaries

Cell SHOULD handle:

- Position.
- Tree storage.
- Local wind exposure.

Cell SHOULD NOT handle:

- Overall forest management.
- Fire-spread calculations.
- Global wind calculations.
- Ignition behaviour.
- GUI drawing.

Shared API – KEEP CONSISTENT

```
Cell(int x, int y)
boolean hasTree()
Tree getTree()
void plantTree(Tree tree)
int getX()
int getY()
double getWindExposure()
void setWindExposure(double windExposure)
```

TREE.JAVA – IMPLEMENTATION SPECIFICATION

Purpose

`Tree` represents one individual tree and stores information about its burning state and how easily it can become involved in a fire.

Class

Class name: `Tree`

Implements: `Burnable`

Fields

`boolean burning`

Whether the Tree is currently burning.

`double spreadability`

How easily the Tree can become involved in fire spreading.

Higher values represent greater spreadability.

Constructor

`Tree(double spreadability)`

Parameter:

`spreadability` – the Tree's fire spreadability.

A new Tree should normally begin in a non-burning state.

Relationships

- Cell HAS a Tree.
- Tree IMPLEMENTS Burnable.

Required Methods

`void ignite()`

Starts the Tree burning.

`void ignite(double severity)`

Attempts to ignite the Tree using a supplied ignition severity.

This is an overloaded version of `ignite()`.

`boolean isBurning()`

Returns whether the Tree is currently burning.

`double getSpreadability()`

Returns the Tree's spreadability.

`void extinguish()`

Stops the Tree from burning.

OOP Concepts

Interface:

Tree implements `Burnable`.

Method overloading:

`ignite()` and `ignite(double severity)`.

Responsibility Boundaries

Tree SHOULD handle:

- Its burning state.
- Its spreadability.
- Being ignited or extinguished.

Tree SHOULD NOT handle:

- Finding neighbouring Trees.
- Managing the forest grid.
- Wind calculations.
- GUI drawing.

Shared API – KEEP CONSISTENT

```
Tree(double spreadability)
void ignite()
void ignite(double severity)
boolean isBurning()
double getSpreadability()
void extinguish()
```

BURNABLE.JAVA – IMPLEMENTATION SPECIFICATION

Purpose

`Burnable` defines the common behaviour required for objects that can burn.

Type

Interface name: `Burnable`

Required Methods

```
void ignite()
```

Starts the object burning.

```
boolean isBurning()
```

Returns whether the object is currently burning.

Relationships

Tree implements `Burnable`.

OOP Concept

This provides the project with an interface.

The interface defines what an object CAN DO without deciding exactly how the object performs that behaviour.

Responsibility Boundaries

Burnable SHOULD define required burning behaviour.

It SHOULD NOT contain Tree-specific, Forest-specific or fire-spreading logic.

Shared API – KEEP CONSISTENT

```
void ignite()
```

```
boolean isBurning()
```

FOREST.JAVA – IMPLEMENTATION SPECIFICATION

Purpose

`Forest` represents the overall forest environment and manages the main simulation state.

It owns the grid of Cells and coordinates fire spreading.

Class

Class name: `Forest`

No inheritance is currently required.

Fields

`Cell[][] grid`

Two-dimensional collection containing the Forest's Cells.

`double temperature`

Current environmental temperature.

`double rainfall`

Current rainfall/moisture factor.

`double aridity`

How dry the environment is.

`Wind wind`

The current Wind affecting the Forest.

A random-number generator may also be used for random generation and probability calculations.

Constructor

`Forest(int rows, int columns)`

Parameters:

`rows` – number of grid rows.

`columns` – number of grid columns.

The Forest should create its Cell grid and may populate Cells with Trees.

Relationships

- Forest HAS many Cells.
- Forest HAS a Wind.
- Cells may contain Trees.

Required Methods

`Cell[][] getGrid()`

Returns the Forest grid.

```
void setWind(Wind wind)
```

Assigns the current Wind to the Forest.

```
Wind getWind()
```

Returns the current Wind.

```
void update()
```

Moves the simulation forward by one step and coordinates fire spreading.

```
void display()
```

Produces a simple representation of the Forest state for testing/display purposes.

Fire-Spreading Responsibility

The Forest should coordinate fire spreading by considering:

- Currently burning Trees.
- Neighbouring Cells.
- Whether neighbouring Cells contain Trees.
- Tree spreadability.
- Wind exposure.
- Aridity.
- Rainfall.
- Probability/randomness.

Responsibility Boundaries

Forest SHOULD handle:

- Overall grid.
- Environmental values.
- Simulation updates.
- Fire-spread coordination.

Forest SHOULD NOT handle:

- GUI drawing.
- Specific Lightning/Arson/Backburning behaviour.
- Tree's internal burning behaviour.
- Swing window management.

Shared API – KEEP CONSISTENT

```
Forest(int rows, int columns)
```

```
Cell[][] getGrid()
```

```
void setWind(Wind wind)
```

```
Wind getWind()  
void update()  
void display()
```

WIND.JAVA – IMPLEMENTATION SPECIFICATION

Purpose

`Wind` represents the wind conditions affecting the Forest.

It stores wind speed and direction and determines how strongly Cells are affected.

Class

Class name: `Wind`

No inheritance is currently required.

Fields

`double speed`

Current wind strength/speed.

`String direction`

Direction in which the wind is travelling.

Possible directions include:

NORTH

SOUTH

EAST

WEST

Constructor

`Wind(double speed, String direction)`

Parameters:

`speed` – wind speed.

`direction` – wind direction.

Relationships

- Forest HAS a Wind.
- Wind affects many Cells.
- Cells store their resulting wind exposure.

Required Methods

`double getSpeed()`

Returns the wind speed.

`String getDirection()`

Returns the wind direction.

`void applyWind(Forest forest)`

Determines how Wind affects the supplied Forest and updates Cell wind exposure.

Responsibility Boundaries

Wind SHOULD handle:

- Wind speed.
- Wind direction.
- Determining local wind influence.

Wind SHOULD NOT handle:

- Tree creation.
- Ignition behaviour.
- Overall simulation timing.
- GUI rendering.

Shared API – KEEP CONSISTENT

`Wind(double speed, String direction)`

`double getSpeed()`

`String getDirection()`

`void applyWind(Forest forest)`

IGNITIONSOURCE.JAVA – IMPLEMENTATION SPECIFICATION

Purpose

`IgnitionSource` represents a general source capable of starting a forest fire.

It is the common parent of Lightning, Arson and Backburning.

Class

Class name: `IgnitionSource`

Type: Abstract class

Fields

`double severity`

How powerful the ignition source is.

`double spreadability`

The ignition source's contribution to fire/spread behaviour.

Constructor

The parent constructor should receive:

`double severity`

`double spreadability`

Child classes provide their appropriate values.

Inheritance

IgnitionSource

→ Lightning

→ Arson

→ Backburning

Required Method

`void ignite(Forest forest)`

Abstract method defining the common action of starting a fire in a Forest.

Each child class provides its own implementation.

OOP Concepts

Inheritance:

Lightning, Arson and Backburning extend IgnitionSource.

Overriding:

Each child provides its own `ignite(Forest forest)`.

Polymorphism:

An `IgnitionSource` reference can represent Lightning, Arson or Backburning.

Responsibility Boundaries

IgnitionSource SHOULD handle:

- Shared ignition properties.
- Common ignition structure.

Specific ignition behaviour belongs to its subclasses.

Shared API – KEEP CONSISTENT

Constructor parameters:

```
double severity  
double spreadability
```

Method:

```
void ignite(Forest forest)
```

LIGHTNING.JAVA – IMPLEMENTATION SPECIFICATION

Purpose

`Lightning` represents a natural lightning strike.

Its main characteristic is that the ignition location is randomly selected.

Class

Class name: `Lightning`

Extends: `IgnitionSource`

Constructor

`Lightning()`

No location is required because Lightning chooses its strike location randomly.

The constructor should provide appropriate Lightning severity/spreadability values to IgnitionSource.

Required Method

`void ignite(Forest forest)`

Overrides the parent method.

General behaviour:

1. Access the Forest grid.
2. Choose a random location.
3. Locate the Cell.
4. Check whether it contains a Tree.
5. Attempt to ignite that Tree using Lightning's severity.

OOP Concepts

- Inheritance.
- Method overriding.
- Polymorphism.

Responsibility Boundaries

Lightning SHOULD handle:

- Random strike selection.
- Lightning-specific ignition.

Lightning SHOULD NOT handle:

- Fire spreading after ignition.
- Wind.
- Forest updates.
- GUI drawing.

Shared API – KEEP CONSISTENT

```
Lightning()  
void ignite(Forest forest)
```

ARSON.JAVA – IMPLEMENTATION SPECIFICATION

Purpose

`Arson` represents a deliberately started fire at a chosen location.

Unlike `Lightning`, its ignition location is supplied rather than randomly selected.

Class

Class name: `Arson`

Extends: `IgnitionSource`

Fields

`int row`
Target row.

`int col`
Target column.

Constructor

```
Arson(int row, int col)
```

Parameters:

`row` – target row.

`col` – target column.

The constructor should also provide appropriate `Arson` severity/spreadability values to `IgnitionSource`.

Required Method

```
void ignite(Forest forest)
```

Overrides the parent method.

General behaviour:

1. Access the Forest grid.
2. Validate the target location.
3. Locate the target Cell.
4. Check whether a Tree exists.
5. Attempt to ignite the Tree using Arson's severity.

OOP Concepts

- Inheritance.
- Method overriding.
- Polymorphism.

Responsibility Boundaries

Arson SHOULD handle:

- Chosen ignition location.
- Arson-specific ignition behaviour.

Arson SHOULD NOT handle:

- General fire propagation.
- Wind.
- Forest generation.
- GUI rendering.

Shared API – KEEP CONSISTENT

```
Arson(int row, int col)
void ignite(Forest forest)
```

BACKBURNING.JAVA – IMPLEMENTATION SPECIFICATION

Purpose

Backburning represents a controlled fire intentionally started across part of the Forest.

Unlike Arson, which targets one location, Backburning affects multiple Trees in a controlled pattern.

The current design uses a selected row.

Class

Class name: `Backburning`

Extends: `IgnitionSource`

Fields

`int row`

The row selected for the controlled burn.

Constructor

`Backburning(int row)`

Parameter:

`row` – row where the controlled burn occurs.

The constructor should provide appropriate Backburning severity/spreadability values to IgnitionSource.

Required Method

`void ignite(Forest forest)`

Overrides the parent method.

General behaviour:

1. Access the Forest grid.
2. Validate the selected row.
3. Move through Cells in that row.
4. Find Cells containing Trees.
5. Attempt to ignite those Trees.

OOP Concepts

- Inheritance.
- Method overriding.

- Polymorphism.

Responsibility Boundaries

Backburning SHOULD handle:

- Controlled ignition pattern.
- Selected row.
- Backburning-specific ignition behaviour.

Backburning SHOULD NOT handle:

- General fire propagation.
- Wind.
- Forest generation.
- GUI drawing.

Shared API – KEEP CONSISTENT

```
Backburning(int row)
void ignite(Forest forest)
```

FORESTPANEL.JAVA – IMPLEMENTATION SPECIFICATION

Purpose

`ForestPanel` provides the graphical visualisation of the Forest.

It reads simulation state and converts it into a visual grid.

Class

Class name: `ForestPanel`

Extends: `JPanel`

Fields

`Forest forest`

Reference to the Forest being displayed.

ForestPanel should display the same Forest object that the simulation is updating.

Constructor

```
ForestPanel(Forest forest)
```

Parameter:

`forest` – Forest to display.

Required Method

```
paintComponent(Graphics g)
```

Used by Swing to draw the simulation.

General behaviour:

1. Obtain the Forest grid.
2. Determine appropriate Cell sizes.
3. Loop through the Cells.
4. Inspect each Cell.
5. Draw the appropriate visual state.

Suggested states:

Empty Cell = dark/black

Living Tree = green

Burning Tree = red

Additional states may be introduced later.

Relationships

ForestPanel reads:

Forest → Cell → Tree

Main creates the ForestPanel and places it inside the application window.

Responsibility Boundaries

ForestPanel SHOULD handle:

- Drawing the Forest.
- Translating simulation state into visual state.

ForestPanel SHOULD NOT handle:

- Fire spreading.
- Ignition.
- Wind calculations.
- Tree creation.

Shared API – KEEP CONSISTENT

```
ForestPanel(Forest forest)
paintComponent(Graphics g)
```

MAIN.JAVA – IMPLEMENTATION SPECIFICATION

Purpose

`Main` is the entry point of the application.

It creates the major objects, connects them together and starts the simulation.

Class

Class name: `Main`

Required Method

```
public static void main(String[] args)
```

Java begins execution here.

Main Responsibilities

Main should:

1. Create the Forest.
2. Create the Wind.
3. Connect Wind to Forest.
4. Create an IgnitionSource.
5. Start the initial fire.
6. Create the graphical window.
7. Create ForestPanel.

8. Add ForestPanel to the window.
9. Start the simulation timer.
10. Repeatedly update the Forest and redraw the GUI.

Main Objects

Main is expected to work with:

- Forest
- Wind
- IgnitionSource
- ForestPanel
- JFrame
- Timer

The IgnitionSource may be:

- Lightning.
- Arson.
- Backburning.

Polymorphism

Main can use an `IgnitionSource` reference while the actual object is Lightning, Arson or Backburning.

This demonstrates polymorphism and allows different ignition types to be substituted.

Simulation Cycle

The simulation should repeatedly perform:

Forest update
→ GUI repaint
→ Forest update
→ GUI repaint

Responsibility Boundaries

Main SHOULD handle:

- Creating major objects.
- Connecting components.
- Starting the application.
- Starting the GUI.

- Simulation timing.

Main SHOULD NOT contain:

- Detailed fire-spread calculations.
- Detailed Tree behaviour.
- Wind calculations.
- Detailed ignition algorithms.
- Cell drawing logic.

Shared API – KEEP CONSISTENT

```
public static void main(String[] args)
```